**23CSE455 Design Patterns**

# PART 1 – DESIGN PATTERN DISCOVERY

*Scientific Research Data Platform using FastAPI and React*

**Case Study Title:** Design Pattern Identification and Evolution in a FastAPI and React-Based Scientific Research Data Platform

**Application:** Data to Science (D2S) — UAV & Field Remote Sensing Repository

**Date:** October 10, 2026

**Repository:** GitHub: https://github.com/gdslab/data-to-science.git (Commit: 02230dd745efef089f1f6c30298f0761c17f530c)

**Team Members**


| Roll number | Student name | Batch |
|---|---|---|
| AM.SC.U4CSE23321 | Harikrishna T P | CSE D |
| AM.SC.U4CSE23313 | C M Chandana | CSE D |
| AM.SC.U4CSE23342 | Pooja T | CSE D |
| AM.SC.U4CSE23345 | Rithu Mithra S S | CSE D |


## 1 Introduction

Software design patterns represent time-tested, reusable architectural and object-oriented abstractions that solve recurring engineering challenges within modern distributed systems. As scientific research increasingly relies on petabyte-scale sensor collections, unmanned aerial vehicle (UAV) remote sensing, and collaborative data-sharing platforms, establishing robust, extensible, and decoupled software architectures becomes critical.

This case study investigates the architectural baseline of **Data to Science (D2S)**, an open-source, cloud-native scientific research platform built with FastAPI, React, PostgreSQL/PostGIS, and Celery. The objective of Part 1 is to investigate baseline capabilities across key functional domains, identify underlying design patterns implemented across the frontend and backend, map them to repository source evidence, formulate comprehensive UML diagrams, and establish the baseline motivating software evolution.

## 2 Case Study Selection

### 2.1 Data to Science (D2S)

**Data to Science (D2S)** is an open-source platform developed by Dr. Jinha Jung's research group at Purdue University (GDSL) and published in the *Journal of Open Source Software (JOSS)* (DOI: 10.21105/joss.09497). The platform is purposefully designed to ingest, process, catalog, visualize, and disseminate multi-gigabyte spatial raster datasets (orthomosaics, digital surface models [DSM]), raw imagery archives, and 3D LiDAR point clouds acquired via UAVs for agricultural and forestry research.

- **Repository URL:** https://github.com/gdslab/data-to-science.git
- **Baseline Git Commit:** `02230dd745efef089f1f6c30298f0761c17f530c`
- **License:** MIT License

### 2.2 Reason for Selecting D2S

D2S was chosen as our case study subject for three key reasons:

1. **Modern Dual-Stack Framework:** Combines an asynchronous Python backend (FastAPI 0.139 / Starlette / SQLAlchemy 2.0) with a modern reactive frontend (React 18 / TypeScript / Vite), providing a rich surface for both backend structural/creational patterns and frontend behavioral/reactive patterns.
2. **Complex Real-World Domain:** Managing multi-temporal UAV sorties, heavy raster calculations, STAC standards, and multi-gigabyte spatial assets presents real engineering challenges (chunked uploads, async workers, security proxies, and interchangeable photogrammetry pipelines).
3. **Clear Architectural Boundaries and Evolution Targets:** The codebase exhibits strong modular boundaries (generic repositories, dependency injection, and security proxies) while demonstrating distinct capability gaps (absence of dataset versioning, absence of fine-grained policy-based access control, and manual workflow orchestration) that motivate software evolution.

## 3 Application Description

### 3.1 Problem Domain

In agricultural phenotyping and forestry canopy modeling, scientists deploy UAVs equipped with multispectral, RGB, and LiDAR sensors. Raw flight sorties produce thousands of high-resolution images that are processed into dense point clouds, digital elevation models (DEM), and orthomosaics. Historically, research teams faced severe friction sharing massive datasets, enforcing SpatioTemporal Asset Catalog (STAC) standards, and managing cross-institutional collaboration. D2S resolves this by providing a unified, cloud-ready repository bridging data ingestion, photogrammetry processing, spatial indexing, and web-based interactive exploration.

### 3.2 Target Users

- **Field Researchers & Pilots:** Log sorties, record sensor parameters (altitude, camera model), and upload raw image archives.
- **Remote Sensing Scientists:** Execute photogrammetry pipelines, extract spatial boundaries, and analyze NDVI/DSM layers.
- **Institutional Collaborators:** Share project assets privately within teams under granular roles (Owner, Manager, Viewer).
- **General Scientific Community:** Access published open-science research datasets without requiring authentication.
- **System Administrators:** Monitor disk usage, approve registrations, and configure external compute worker nodes.

### 3.3 Functional Features

- **Resumable Chunked Uploads:** High-performance, fault-tolerant ingestion of multi-gigabyte archives via the Tus protocol (tusd).

- **Hierarchical Research Organization:** Hierarchical grouping of assets into Projects, seasonal Field Campaigns, and Flights.
- **Photogrammetry Cloud Integration:** Triggering automated orthomosaic and 3D point cloud generation via external compute engines (Agisoft Metashape and OpenDroneMap).
- **Interactive Geospatial Visualization:** Integrated OpenLayers/MapLibre map views and Potree/PlayCanvas 3D WebGL point cloud viewers.
- **STAC Catalog Compliance:** Automatic extraction of SpatioTemporal Asset Catalog metadata, spatial bounding boxes, and Coordinate Reference Systems (CRS) using GDAL and Rasterio.
- **Secure File Serving:** Dynamic authorization checks for static geospatial assets with byte-range request streaming.

### 3.4 Technology Stack

| Layer | Technologies | Version / Notes |
|---|---|---|
| Backend Framework | FastAPI, Starlette, Uvicorn | Python 3.12, FastAPI 0.139.0, Starlette 1.3.1 |
| Data Persistence | PostgreSQL, PostGIS, SQLAlchemy, Alembic | PostgreSQL 16, SQLAlchemy 2.0.51, Alembic 1.18.5 |
| Geospatial | GDAL, Rasterio, Fiona, GeoPandas, PDAL | GDAL 3.12.3, Rasterio 1.5.0, PDAL 3.5.3 |
| Task Queue | Celery, Redis, RabbitMQ | Celery 5.5.3, Redis 5.0+ |
| Frontend | React, TypeScript, Vite, React Router | React 18, TypeScript 5, Vite 5, React Router 7 |
| UI & Styling | TailwindCSS, Material-UI (MUI), Headless UI | Component-driven styling and responsive design |
| Renderers | MapLibre GL, OpenLayers, Potree, PlayCanvas | WebGL spatial tile and 3D point cloud streaming |
| Upload & Proxy | tusd, MinIO (S3), NGINX, Varnish | Chunked resumable tus protocol and tile caching |

## 4 Framework Architecture

### 4.1 Frontend

The frontend is a single-page application built with React 18 and Vite. It employs declarative client-side routing (`frontend/src/router.tsx`) with lazy loading and route guards (`RequireAuth`, `RequireAdmin`). An application-wide `AuthContext.tsx` provider maintains user profile state. A customized Axios HTTP client (`frontend/src/api.ts`) utilizes response interceptors to buffer failed requests and execute token refreshes transparently.

### 4.2 Backend

The backend leverages FastAPI structured across modular layers:

- **Router Layer** (`app/api/api_v1/endpoints/`): RESTful endpoints for projects, flights, data products, raw uploads, and STAC catalogs.

- **Dependency Injection Framework** (`app/api/deps.py`): Supplies request-scoped database sessions (`get_db`) and user authentication validators.
- **CRUD Repository Layer** (`app/crud/`): Encapsulates database transactions behind reusable generic repository classes derived from `CRUDBase`.
- **Security Interceptors:** Custom file proxy (`ProtectedStaticFiles.py`) enforcing byte-range access control.

### 4.3 Database

PostgreSQL with PostGIS extensions stores relational entities and spatial geometries. SQLAlchemy 2.0 models declare foreign keys, cascading deletes, and JSONB spatial column attributes (`bbox`, `crs`, `resolution`, `stac_properties`). Schema migrations are tracked with Alembic.

### 4.4 Processing/Task Layer

Celery workers backed by Redis handle long-running background tasks (`backend/app/tasks/`), including point cloud preview rendering, STAC collection builds, and disk usage calculations. The `RpcClient` utility interfaces with external compute nodes running photogrammetry algorithms (Agisoft Metashape or OpenDroneMap).

### 4.5 Data Flow

1. **Ingestion Flow:** Researcher uploads raw imagery chunks via Tusd through NGINX. Upon completion, Tusd triggers a FastAPI webhook to record `RawData`.
2. **Processing Flow:** FastAPI dispatches a Celery task, invoking `RpcClient` with the selected photogrammetry strategy. Upon completion, generated orthomosaics and point clouds are cataloged as `DataProduct` entities.
3. **Serving Flow:** Static geospatial assets requested by React viewports are intercepted by `ProtectedStaticFiles`, verified against database permissions, and streamed using HTTP 206 Partial Content range requests.

## 5 Functional Module Investigation

| Functional Module | Status | Evaluation & Repository Evidence |
|---|---|---|
| Researcher Management | Present | Full user lifecycles, institutional affiliations, research teams, and collaboration roles (OWNER, MANAGER, VIEWER) verified in `models/user.py`, `models/team.py`, and `models/project_member.py`. |
| Dataset Management | Present | Ingestion and management of raw archives, orthomosaics, DSM, point clouds, and vector shapefiles via chunked Tus uploads verified in `models/data_product.py` and `endpoints/tusd.py`. |
| Experiment / Processing | Present | Hierarchical experimental structuring (Project → Campaign → Flight) and photogrammetry RPC dispatch verified in `models/project.py`, `models/flight.py`, and `tasks/raw_image_processing_tasks.py`. |

| Functional Module | Status | Evaluation & Repository Evidence |
|---|---|---|
| Metadata | Present | Automated spatial extraction (CRS, BBox, resolution via Rasterio), STAC compliance, and spatial annotations verified in `models/data_product_metadata.py` and `tasks/stac_tasks.py`. |
| Access Control | Partial | Static project roles and binary public toggles exist. However, Attribute-Based Access Control (ABAC), embargo periods, and policy rule engines are absent. (Motivates Evolution 2). |
| Workflow | Partial | Celery executes ad-hoc worker tasks and RPC calls. However, automated multi-step DAG experiment orchestration is absent. (Motivates Evolution 3). |
| Notifications | Partial / cannot fully confirm | We found transactional email paths and workflow listener hooks, but we could not establish a complete notification center or deployed WebSocket delivery from our supplied evidence. |

# 6 Design Pattern Discovery

## 6.1 Pattern Identification Method

Patterns were identified through static AST inspection, inheritance hierarchy tracing, runtime dependency flow analysis, and Axios interceptor examination. All candidates were verified against concrete code symbols, while non-pattern implementations (such as non-polymorphic entity constructors) were explicitly rejected.

## 6.2 Pattern Inventory

| Pattern Type | Instance Collaboration Name | Instances | Concrete Classes |
|---|---|---|---|
| Repository | Data Product Entity Repository | 1 | 2 (`CRUDBase`, `CRUDDataProduct`) |
| Dependency Injection | Request Session and Auth Context Injection | 1 | 2 (`get_db`, `get_current_approved_user`) |
| Proxy / Decorator | Static Asset Authorization Proxy | 1 | 2 (`RangedStaticFiles`, `ProtectedStaticFiles`) |
| Strategy | Photogrammetry Reconstruction Strategy | 1 | 2 (`MetashapeQueryParams`, `ODMQueryParams`) |
| Chain of Responsibility | Axios Interceptor Token Refresh Pipeline | 1 | 1 (`api.interceptors.response`) |
| Observer | React Authentication State Distribution | 1 | 2 (`AuthContextProvider`, `RequireAuth`) |

# 7 Pattern-to-Source-Code Mapping

1. **Repository Pattern (Data Product Entity Repository):**
   - *RepositoryBase:* `CRUDBase` (`backend/app/crud/base.py:16-82`)
   - *ConcreteRepository:* `CRUDDataProduct` (`backend/app/crud/crud_data_product.py:69-995`)

- *DomainEntity:* `DataProduct` (`backend/app/models/data_product.py:25-109`)
- *Client:* `endpoints/data_products.py` (`backend/app/api/api_v1/endpoints/data_products.py:100-250`)
- *Description:* Encapsulates database queries behind a generic CRUD interface with PostGIS bounding box calculations.

2. **Dependency Injection Pattern (Session & Auth Injection):**
   - *FrameworkInjector:* `fastapi.Depends` (FastAPI runtime)
   - *Providers:* `get_db()` (`app/api/deps.py:37-60`), `get_current_approved_user()` (`app/api/deps.py:111-118`)
   - *DependentRoute:* `create_project()` (`app/api/api_v1/endpoints/projects.py:19-30`)
   - *Description:* Injects request-scoped DB sessions and security credentials into endpoint parameter signatures.

3. **Proxy / Decorator Pattern (Static Asset Authorization Proxy):**
   - *TargetComponent:* `RangedStaticFiles` (`app/utils/staticfiles.py`)
   - *ProxyDecorator:* `ProtectedStaticFiles` (`app/utils/ProtectedStaticFiles.py:568-685`)
   - *AuthVerifier:* `verify_static_file_access()` (`app/utils/ProtectedStaticFiles.py:320-430`)
   - *Description:* Intercepts static file requests to verify JWT/API-key permissions and dynamically compiles Parquet vector layers before streaming.

4. **Strategy Pattern (Photogrammetry Engine Strategy):**
   - *Context:* `transfer_raw_data()` (`app/tasks/raw_image_processing_tasks.py:106-220`)
   - *StrategySelector:* `backend: Literal["metashape", "odm"]` (`tasks/raw_image_processing_tasks.py:120`)
   - *ConcreteStrategies:* `MetashapeQueryParams` (`schemas/raw_data.py:76-98`), `ODMQueryParams` (`schemas/raw_data.py:100-105`)
   - *Description:* Modularizes photogrammetry parameterization into interchangeable strategies for Metashape and OpenDroneMap.

5. **Chain of Responsibility / Interceptor Pattern:**
   - *Pipeline:* `api.interceptors.response.use()` (`frontend/src/api.ts:27-77`)
   - *QueueManager:* `processQueue()` & `failedQueue` (`frontend/src/api.ts:13-25`)
   - *Description:* Intercepts HTTP 401 Unauthorized errors, queues outbound requests, executes token refresh, and replays requests.

6. **Observer Pattern (React Authentication State Distribution):**
   - *SubjectProvider:* `AuthContextProvider` (`frontend/src/AuthContext.tsx:51-162`)
   - *Observers:* `RequireAuth` (`frontend/src/AuthContext.tsx:164-180`), `RequireAdmin` (182-195)
   - *EventBus:* `window.dispatchEvent()` (`frontend/src/api.ts:66`)
   - *Description:* Broadcasts authentication lifecycle transitions across the React virtual DOM tree reactively.

# 8 UML Diagrams

## 8.1 Use Case Diagram

The Use Case Diagram depicts the core user roles (Researcher, Collaborator, Administrator, External Processing Worker) interacting with the D2S platform boundaries:

- **Researchers:** Create projects, record flight sorties, upload raw UAV data via Tus, select photogrammetry strategies, and manage collaborators.
- **Collaborators:** Explore interactive maps, download protected data products, and annotate spatial layers.
- **Administrators:** Review user account approvals and inspect node storage analytics.
- **External Processing Cluster:** Consumes dispatched photogrammetry jobs and produces calibrated spatial products.

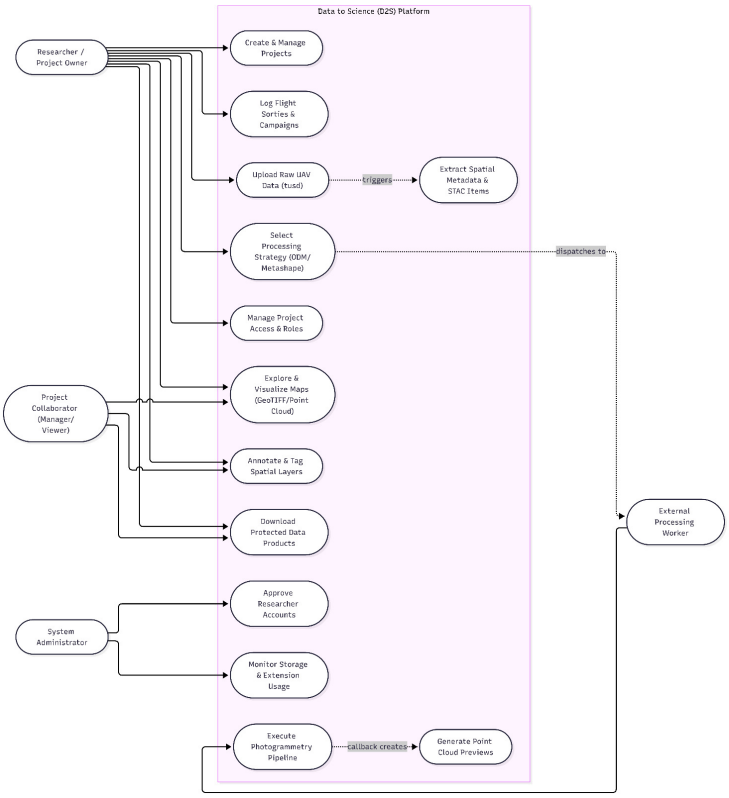



Figure 1: Actors and use cases of the Data to Science platform.

## 8.2 Component Diagram

The architecture and runtime component boundaries are presented in Figure 2.

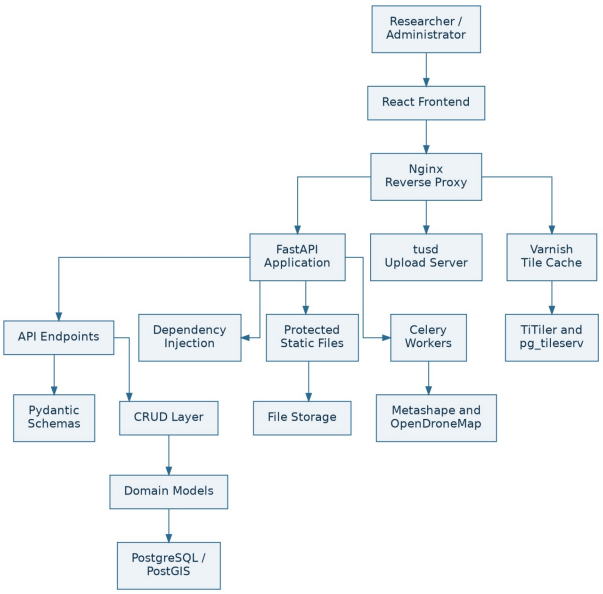



Figure 2: D2S Baseline Framework Architecture and Component Diagram

The diagram models the flow from the client browser and GIS desktop clients through the NGINX reverse proxy to the FastAPI backend, Tusd upload server, and Varnish tile cache. Within FastAPI, requests traverse API endpoints, dependency injection, and CRUD repository layers to PostgreSQL/PostGIS. Asynchronous tasks dispatch via Celery to external Metashape/ODM compute engines, while static file requests are authorized via `ProtectedStaticFiles`.

### 8.3 Class Diagram

The object-oriented structures and participant collaborations of the six discovered design patterns are shown in Figure 3.

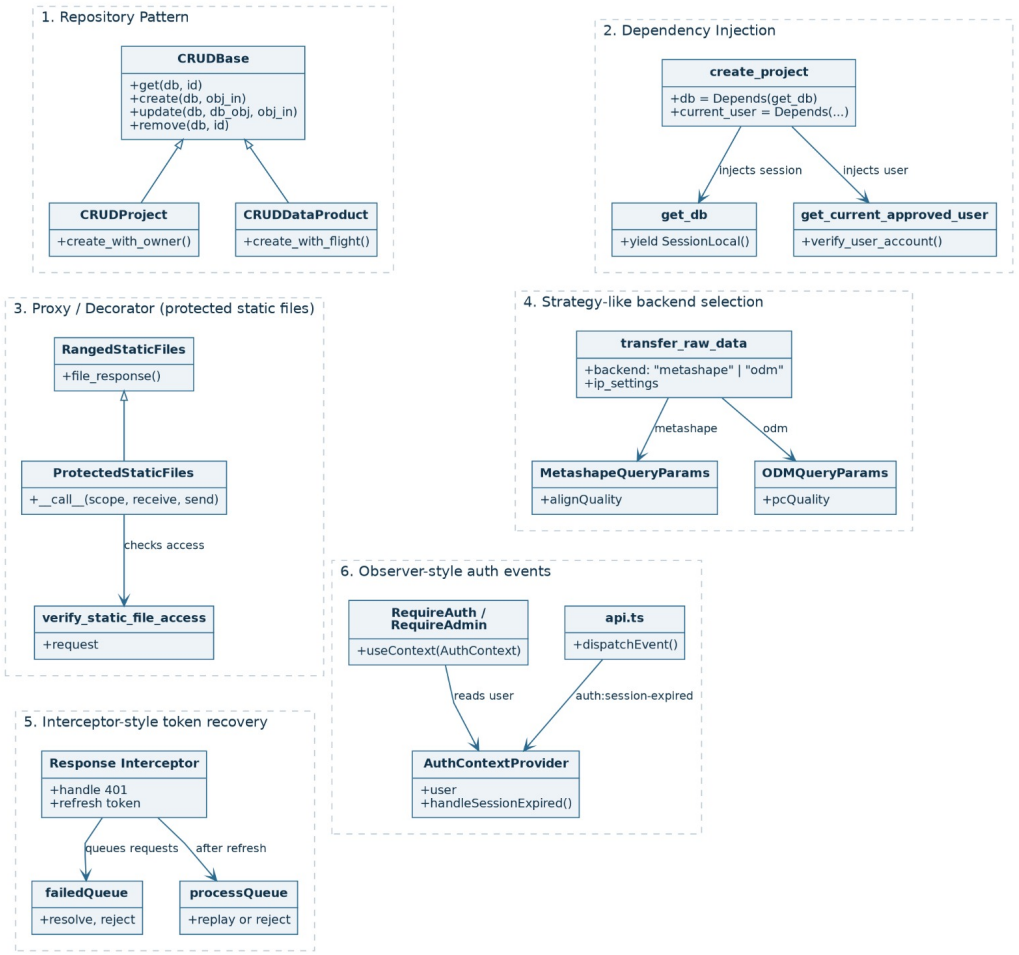



Figure 3: D2S Design Pattern Inventory and Collaboration Class Diagram

The diagram captures the six pattern collaborations: (1) Repository pattern inheritance from `CRUDBase`, (2) Dependency Injection of database sessions and user verification, (3) Proxy/Decorator interception via `ProtectedStaticFiles`, (4) Strategy-based photogrammetry parameters, (5) Interceptor-style token recovery pipeline, and (6) Observer-style authentication state propagation.

## 8.4 Sequence Diagram

The end-to-end operational sequence for raw drone imagery ingestion, strategy execution, and protected streaming is structured as follows:

1. **Chunked Ingestion:** Researcher streams binary chunks to Tusd via NGINX. Tusd completes assembly and triggers the FastAPI webhook.
2. **Dependency Injection & Ingestion:** FastAPI injects the session and user, creates the `RawData` database record via the repository layer, and enqueues a Celery task.
3. **Strategy Execution:** Celery selects either `MetashapeQueryParams` or `ODMQueryParams` and transfers data to the RPC cluster. The resulting orthomosaics and point clouds are persisted into `DataProduct`.
4. **Protected Delivery:** Map requests to `/static/...` are intercepted by `ProtectedStaticFiles`, verified against project permissions, and streamed using HTTP 206 Partial Content.

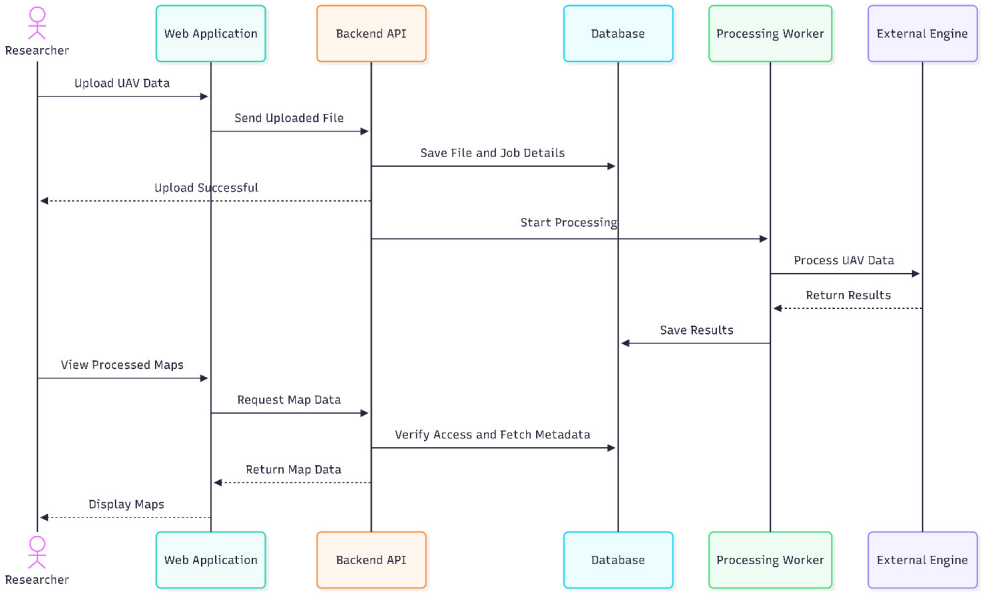



Figure 4: Upload, asynchronous processing and map retrieval.

## 9 Summary of Part 1

The Part 1 pattern discovery on Data to Science (D2S) establishes a robust architectural baseline:

- **Strengths:** The system effectively separates persistence concerns via the Repository Pattern, decouples session lifecycles through Dependency Injection, secures multi-gigabyte raster files using a Security Proxy, enables modular photogrammetry via the Strategy Pattern, and maintains seamless frontend sessions with Interceptors and Observers.
- **Motivation for Part 2 Evolution:**
  1. **Dataset Versioning is Absent:** DataProduct entities are unversioned singletons. Overwriting data destroys history. This motivates Evolution Scenario 1 (CH01): Add dataset versioning.
  2. **Access Control is Coarse-Grained:** Lacks attribute-based policies, embargo time-locks, and dynamic rule engines. This motivates Evolution Scenario 2 (CH02): Add advanced access-control policies.
  3. **Workflow Orchestration is Manual:** Background tasks are unlinked without DAG sequencing. This motivates Evolution Scenario 3 (CH03): Add experiment workflow automation.

## 10 References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
3. Jung, J., et al. (2024). Data to Science: An Open-Source Platform for UAV Remote Sensing Data Sharing. *Journal of Open Source Software (JOSS)*, DOI: 10.21105/joss.09497.
4. FastAPI Documentation. Dependencies - First Steps. https://fastapi.tiangolo.com/tutorial/dependencies/.

5. Tus.io. Tus Resumable Upload Protocol v1.0.0. https://tus.io/protocols/resumable-upload.
6. SpatioTemporal Asset Catalog (STAC) Specification. https://stacspec.org/.
7. D2S GitHub Repository: https://github.com/gdslab/data-to-science.